ML Programs

Exercise - 1:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
print(df.head())

import pandas as pd
# Read Iris CSV file
df = pd.read_csv("iris.csv")
# Display first 10 rows
print(df.head(10))

print("Shape:", df.shape)
print(df.columns)
print(df.info())
print(df)
print(df.dtypes)

print(df.describe())

print(df.isnull().sum())

print(df.duplicated().sum())

df.boxplot(figsize=(8,5))
plt.show()

import matplotlib.pyplot as plt
df.hist(figsize=(10,8))
plt.show()

plt.scatter(df['SepalLengthCm'],
 df['PetalLengthCm'])
plt.xlabel('Sepal Length')
plt.ylabel('Petal Length')
plt.show()
```

```
sns.pairplot(df)
plt.show()

corr = df.corr(numeric_only=True)
print(corr)

import seaborn as sns
corr = df.iloc[:, :-1].corr()
sns.heatmap(corr, annot=True,cmap='coolwarm')
plt.show()

sns.boxplot(data=df)
plt.show()

sns.countplot(x='Species', data=df)
plt.show()
```

Exercise - 2:

```
import numpy as np
# Step 1: Create the data
x = np.array([10, 20, 30, 40, 50])
y = np.array([15, 25, 35, 45, 55])
# Probability values for Expected Value
p = np.array([0.1, 0.2, 0.3, 0.2, 0.2])
# a. Expected Value
# Formula: E(X) = Σ (x × p)
expected_value = np.sum(x * p)
print("Expected Value =", expected_value)
# b. Mean
# Formula: Mean = Σx / n
mean = np.mean(x)
print("Mean =", mean)
# c. Standard Deviation
# Formula: σ = √( Σ(x-μ)² / n )
std = np.std(x)
print("Standard Deviation =", std)
# d. Variance
# Formula: σ² = Σ(x-μ)² / n
variance = np.var(x)
print("Variance =", variance)
# e. Covariance
# Formula:
```

```
# Cov(X,Y) = Σ[(Xi-X̄)(Yi-Ȳ)] / (n-1)
covariance = np.cov(x, y)[0, 1]
print("Covariance =", covariance)
# f. Covariance Matrix
# Matrix containing variances and covariance values.
#
cov_matrix = np.cov(x, y)
print("Covariance Matrix =")
print(cov_matrix)
```

Exercise - 3:

```
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
data=pd.read_csv("Employee_Data.csv")
print(data.isnull().sum())
imp=SimpleImputer(strategy='mean')
data[['Age','Salary']]=imp.fit_transform(data[['Age','Salary']])
Q1=data['Salary'].quantile(0.25);Q3=data['Salary'].quantile(0.75)
IQR=Q3-Q1
outliers=data[(data['Salary']<Q1-1.5*IQR)|(data['Salary']>Q3+1.5*IQR)]
X=data.iloc[:,0:3].values
y=data.iloc[:,3].values
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_st
ate=0)
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(X_test)
```

Exercise - 4:

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

dataset = pd.read_csv("Categorical_Data.csv")
```

```
print("Original Dataset")
print(dataset)

print("\nMissing Values")
print(dataset.isnull().sum())

imputer = SimpleImputer(strategy='mean')
dataset[['Age','Salary']] = imputer.fit_transform(dataset[['Age','Salar
y']])
print("\nDataset After Replacing Missing Values")
print(dataset)

X = dataset.iloc[:,0:4]
y = dataset.iloc[:,4]

ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0,3])],
remainder='passthrough')
X = ct.fit_transform(X)

le = LabelEncoder()
y = le.fit_transform(y)

print("\nEncoded Features")
print(X)
print("\nEncoded Target")
print(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=42)
print("\nTraining Samples :", len(X_train))
print("Testing Samples :", len(X_test))

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

print("\nScaled Training Data\n")
print(X_train)
print("\nScaled Testing Data\n")
print(X_test)
```

Exercise - 5:

```
# Decision Tree using ID3-style criterion
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
# Load the Play Tennis dataset
data = pd.read_csv("play_tennis.csv")
X = data.drop(columns=["Play"])
y = data["Play"]
# Convert categorical attributes to dummy variables
X_encoded = pd.get_dummies(X)
# Split the data
X_train, X_test, y_train, y_test = train_test_split( X_encoded, y, test_si
ze=0.30, random_state=42, stratify=y)
# ID3 uses entropy as the splitting criterion
model = DecisionTreeClassifier(criterion="entropy",random_state=42)
model.fit(X_train, y_train)
# Predict test samples
y_pred = model.predict(X_test)
print("Decision Tree Rules:")
print(export_text(model, feature_names=list(X_encoded.columns)))
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
# Classify a new sample
new_sample = pd.DataFrame([{ "Outlook": "Sunny", "Temperature": "Cool", "H
umidity": "Normal",
"Wind": "Strong"}])
new_encoded = pd.get_dummies(new_sample)
new_encoded = new_encoded.reindex(columns=X_encoded.columns, fill_value=0)
prediction = model.predict(new_encoded)
print("New Sample:", new_sample.iloc[0].to_dict())
print("Predicted class:", prediction[0])
# Display the decision tree
plt.figure(figsize=(14, 8))
plot_tree(model, feature_names=X_encoded.columns, class_names=model.classe
s_, filled=True)
plt.title("Decision Tree using ID3-style Entropy Criterion")
plt.show()
```

Exercise - 6:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score
# Load dataset
data=pd.read_csv('Salary_Data_ML.csv')
# X = Independent variable
X=data[['YearsExperience']]
# y = Dependent variable
y=data['Salary']
# Split dataset into training and testing
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_st
ate=0)
# Create Linear Regression model
model=LinearRegression()
# Train the model
model.fit(X_train,y_train)
# Display intercept and slope
print(model.intercept_)
print(model.coef_[0])
# Predict test data
y_pred=model.predict(X_test)
# Compare Actual vs Predicted
print(pd.DataFrame({'Actual':y_test,'Predicted':y_pred}))
# Evaluate model
print(mean_squared_error(y_test,y_pred))
print(r2_score(y_test,y_pred))
# Plot regression line
plt.scatter(X_train,y_train)
plt.plot(X_train,model.predict(X_train),color='red')
plt.show()
```

Exercise - 7:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# Load dataset
data=pd.read_csv('Salary_Data_ML.csv')
# X = Multiple independent variables
X=data[['YearsExperience','Age','EducationLevel']]
# y = Dependent variable
y=data['Salary']
# Split dataset
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_st
ate=0)
```

```
# Create Multiple Linear Regression model
model=LinearRegression()
# Train the model
model.fit(X_train,y_train)
# Display intercept
print(model.intercept_)
# Display coefficient of each feature
print(pd.DataFrame({'Feature':X.columns,'Coefficient':model.coef_}))
# Predict test data
y_pred=model.predict(X_test)
# Compare Actual vs Predicted
print(pd.DataFrame({'Actual':y_test.values,'Predicted':y_pred}))
# Evaluate model
print(mean_squared_error(y_test,y_pred))
print(r2_score(y_test,y_pred))
# Plot Actual vs Predicted
plt.scatter(y_test,y_pred)
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.show()
```

Exercise - 8:

```
# Logistic Regression for binary classification
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix, classificat
ion_report)
# Load dataset
data = pd.read_csv("student_pass.csv")
X = data[["Hours_Studied", "Attendance"]]
y = data["Pass"]
# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42, stratify=y)
# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Create and train Logistic Regression model
model = LogisticRegression(random_state=42)
model.fit(X_train_scaled, y_train)
```

```
# Predictions
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1]
print("Predicted probabilities:", y_prob)
print("Predicted classes:", y_pred)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
# Classify a new student
new_student = pd.DataFrame({ "Hours_Studied": [4], "Attendance": [72]})
new_scaled = scaler.transform(new_student)
probability = model.predict_proba(new_scaled)[0, 1]
prediction = model.predict(new_scaled)[0]
print("\nNew Student:", new_student.iloc[0].to_dict())
print("Probability of passing:", round(probability, 4))
print("Predicted class:", "Pass" if prediction == 1 else "Fail")
# Optional visualization
plt.scatter(data["Hours_Studied"], data["Attendance"], c=data["Pass"])
plt.xlabel("Hours Studied")
plt.ylabel("Attendance")
plt.title("Student Pass/Fail Dataset")
plt.show()
```

Exercise - 9:

```
# Import libraries required for data handling and plotting
import pandas as pd
import matplotlib.pyplot as plt
# Import clustering and preprocessing tools from scikit-learn
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# ---------------- K-MEANS CLUSTERING ----------------
# Read the K-Means dataset from the CSV file
data = pd.read_csv("kmeans_data.csv")
# Select the numerical columns used for clustering
X = data[["Study_Hours", "Marks"]]
# Create a K-Means model with 2 clusters
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
# Train the model and obtain cluster labels
data["KMeans_Cluster"] = kmeans.fit_predict(X)
# Display the cluster assignments
print("K-Means Cluster Assignments:")
```

```
print(data)
# Plot the two clusters
plt.scatter(X["Study_Hours"], X["Marks"], c=data["KMeans_Cluster"])
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("K-Means Clustering")
plt.show()
# ---------------- HIERARCHICAL CLUSTERING ----------------
# Read the hierarchical clustering dataset
hdata = pd.read_csv("hierarchical_data.csv")
# Select the numerical attributes
HX = hdata[["Annual_Income", "Spending_Score"]]
# Create an agglomerative hierarchical clustering model
hc = AgglomerativeClustering(n_clusters=3, linkage="ward")
# Fit the model and store the cluster labels
hdata["Cluster"] = hc.fit_predict(HX)
# Display the hierarchical clusters
print("\nHierarchical Cluster Assignments:")
print(hdata)
# Plot hierarchical clusters
plt.scatter(HX["Annual_Income"], HX["Spending_Score"], c=hdata["Cluster"])
plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Hierarchical Clustering")
plt.show()
# ---------------- PCA ----------------
# Read the PCA dataset
pdata = pd.read_csv("pca_data.csv")
# Select all numerical features
PX = pdata[["Maths", "Physics", "Chemistry", "Biology"]]
# Standardize the features so all subjects have comparable scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(PX)
# Create PCA to reduce four dimensions to two principal components
pca = PCA(n_components=2)
# Transform the standardized data into two principal components
X_pca = pca.fit_transform(X_scaled)
# Display explained variance information
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)
# Plot the two principal components
plt.scatter(X_pca[:, 0], X_pca[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA - 2D Representation")
plt.show()
```

Exercise - 10:

```
# Experiment 10: KNN and SVM Classification
# Import Pandas for reading the CSV file
import pandas as pd
# Import train_test_split to divide data into training and testing sets
from sklearn.model_selection import train_test_split
# Import StandardScaler to standardize numerical features
from sklearn.preprocessing import StandardScaler
# Import KNN and SVM classifiers
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
# Import accuracy_score to measure prediction accuracy
from sklearn.metrics import accuracy_score
# Read the dataset
data = pd.read_csv("knn_svm_data.csv")
# Select input features
X = data[["Height", "Weight"]]
# Select the target class
y = data["Class"]
# Divide data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42, stratify=y)
# Create a scaler using only the training data
scaler = StandardScaler()
# Fit the scaler on training features and transform them
X_train_scaled = scaler.fit_transform(X_train)
# Transform test features using the same scaler
X_test_scaled = scaler.transform(X_test)
# ---------------- KNN ----------------
# Create a KNN classifier with K = 3
knn = KNeighborsClassifier(n_neighbors=3)
# Train the KNN model
knn.fit(X_train_scaled, y_train)
# Predict classes for test samples
knn_pred = knn.predict(X_test_scaled)
# Calculate KNN accuracy
knn_accuracy = accuracy_score(y_test, knn_pred)
print("KNN Predictions:", knn_pred)
print("KNN Accuracy:", knn_accuracy)
# ---------------- SVM ----------------
# Create an SVM classifier using a linear kernel
svm = SVC(kernel="linear", random_state=42)
# Train the SVM model
svm.fit(X_train_scaled, y_train)
# Predict classes for test samples
svm_pred = svm.predict(X_test_scaled)
# Calculate SVM accuracy
```

```
svm_accuracy = accuracy_score(y_test, svm_pred)
print("\nSVM Predictions:", svm_pred)
print("SVM Accuracy:", svm_accuracy)
```

Exercise - 11:

```
# Experiment 11: Naive Bayes Classifier
# Import Pandas for reading CSV files
import pandas as pd
# Import train_test_split for optional splitting
from sklearn.model_selection import train_test_split
# Import the categorical Naive Bayes classifier
from sklearn.naive_bayes import CategoricalNB
# Import OrdinalEncoder to convert text categories into numbers
from sklearn.preprocessing import OrdinalEncoder
# Import accuracy_score for calculating classification accuracy
from sklearn.metrics import accuracy_score
# Read the training and testing datasets
train_data = pd.read_csv("naive_bayes_train.csv")
test_data = pd.read_csv("naive_bayes_test.csv")
# List the input feature columns
features = ["Outlook", "Temperature", "Humidity", "Wind"]
# Separate training features and target
X_train = train_data[features]
y_train = train_data["Play"]
# Separate testing features and target
X_test = test_data[features]
y_test = test_data["Play"]
# Create an encoder for categorical values
encoder = OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value
=-1)
# Convert training text categories to numeric categories
X_train_encoded = encoder.fit_transform(X_train)
# Convert testing categories using the same encoder
X_test_encoded = encoder.transform(X_test)
# CategoricalNB expects non-negative integer category values.
# Convert encoded values to integers.
X_train_encoded = X_train_encoded.astype(int)
X_test_encoded = X_test_encoded.astype(int)
# Create the Naive Bayes classifier
model = CategoricalNB()
# Train the classifier using the training dataset
model.fit(X_train_encoded, y_train)
# Predict the class of the test records
predictions = model.predict(X_test_encoded)
```

```
# Calculate the accuracy by comparing predictions with actual labels
accuracy = accuracy_score(y_test, predictions)
# Display actual and predicted results
result = pd.DataFrame({ "Actual": y_test.values, "Predicted": prediction
s})
print("Test Results:")
print(result)
print("\nAccuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
```

Exercise - 12:

```
# Experiment 12: Random Forest Classification
# Import Pandas for reading the dataset
import pandas as pd
# Import train_test_split to create training and testing data
from sklearn.model_selection import train_test_split
# Import RandomForestClassifier
from sklearn.ensemble import RandomForestClassifier
# Import accuracy_score and classification_report
from sklearn.metrics import accuracy_score, classification_report
# Read the Random Forest dataset
data = pd.read_csv("random_forest_data.csv")
# Select numerical input features
X = data[["Age", "Salary", "Experience"]]
# Select the target column
y = data["Purchased"]
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.25,
random_state=42, stratify=y)
# Create a Random Forest with 100 decision trees
rf = RandomForestClassifier( n_estimators=100, random_state=42)
# Train the Random Forest model
rf.fit(X_train, y_train)
# Predict the classes of test records
y_pred = rf.predict(X_test)
# Calculate the accuracy
accuracy = accuracy_score(y_test, y_pred)
# Display predictions and actual values
print("Actual Values: ", list(y_test))
print("Predicted Values: ", list(y_pred))
# Display accuracy
print("\nRandom Forest Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
# Display precision, recall and F1-score
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```